Employee Tracking System (ETS)

Windows Desktop Application

Technical Specification & Design Document

Amaze Internet Services Pvt. Ltd.

Version: v1.0 – Draft

Document Status: Internal & Confidential

Prepared By
Shailabh Suman

Reviewed By
Sujeet Kumar
Saurabh Suman

Developer
Ashutosh Kumar

Date
15th November 2025

Contents

13.3 Additional Diagrams

13.3.1 Screenshot Capture Workflow (Extended)

13.3.2 Idle Detection Workflow (Extended)

13.3.3 Upload Workflow (Extended)

13.4 Change Log

1. Introduction

1.1 Purpose of the Document

This document defines the complete technical specification, architecture, design, and functional scope of the Employee Tracking System (ETS) desktop application. It provides developers, reviewers, and stakeholders with a clear understanding of how the system will be built, how it will operate, and what requirements it must satisfy. The document acts as the central reference throughout development, testing, and future maintenance.

1.2 Scope of the Project

The Employee Tracking System (ETS) is a Windows-based desktop application designed to monitor and record employee activity during their working hours. The system captures screenshots at random intervals, checks user activity/idle status, stores locally encrypted data, and syncs data to the server based on configured intervals. ETS focuses strictly on productivity tracking and does not include keystroke logging, screen recording, or audio/video capture.

The scope covers:

- Desktop client (Windows)
- Random screenshot capture
- Idle time detection
- Local encrypted data storage
- Periodic upload of logs and screenshots
- User authentication
- Shift-wise tracking logic
- System tray control and background operation

Mobile application support and macOS/Linux clients are excluded from the current version.

1.3 Intended Audience

This document is intended for:

- Development team (primary: Aushutosh Kumar)
- Reviewers (HR and CEO)
- Internal stakeholders and management
- QA and testing teams
- Future developers or maintainers handling updates or support

1.4 Document Structure

The document is organized into major sections covering requirements, architecture, UI design, data model, API specification, development plan, testing, deployment, and maintenance. Each section provides progressively detailed information to support technical understanding and implementation.

1.5 Definitions and Acronyms

Term / Acronym	Description
ETS	Employee Tracking System
GUI	Graphical User Interface
API	Application Programming Interface
DB	Database
AES	Advanced Encryption Standard
IST	Indian Standard Time – the timezone used for all timestamps, logs, and scheduling
User Logs	Encrypted activity data collected from the client
Screenshot Event	A randomly captured image of the employee screen

2. System Overview

2.1 Project Summary

The Employee Tracking System (ETS) is a Windows desktop application designed to record employee productivity during active work hours. It runs quietly in the background, captures screenshots at random intervals, detects idle time, encrypts all collected data, and syncs it to the server either immediately or at fixed upload intervals. The goal is to provide reliable work-hour insights without invading privacy or collecting unnecessary personal data.

The application is lightweight, stable, secure, and optimized for continuous use on standard office machines.

2.2 Key Objectives

The primary objectives of ETS are:

- Monitor employee activity during assigned work shifts
- Capture screenshots at unpredictable random intervals
- Track idle and active time accurately
- Store all data encrypted on the local system
- Sync screenshots and logs to the server automatically
- Operate silently with minimal performance impact
- Run continuously even if minimized or hidden to the system tray
- Maintain accuracy, consistency, and security at all times
- Provide a clean and simple user interface

Privacy boundaries are respected: no keystroke monitoring, no screen recording, and no audio or video collection.

2.3 High-Level Feature Summary

The system includes the following core features:

- **Random Screenshot Capture**
  Screenshots are taken at variable intervals to prevent predictability.

- **Idle Time Detection**
  Detects keyboard and mouse inactivity to track precise idle duration.
- **Local Encrypted Storage**
  All screenshots and logs are encrypted using AES before being saved.
- **Automated Upload System**
  Data is uploaded to the server either instantly or at intervals (e.g., every 1 hour).
- **Shift-Based Tracking**
  Recording starts and ends based on assigned shift schedules.
- **System Tray Integration**
  ETS runs in the tray, providing non-intrusive continuous operation.
- **Employee Login & Authentication**
  Ensures each device is linked to the correct employee ID.
- **Configurable Settings**
  Settings such as screenshot interval, upload frequency, local storage path, etc.

2.4 Target Platform (Windows)

The current version of ETS is developed exclusively for:

- **Operating System:** Windows 10 and Windows 11 (64-bit)
- **Runtime:** Python-based desktop client (using a GUI framework such as PyQt / PySide / Tkinter – finalized in the architecture section)
- **Storage:** Local disk (encrypted)
- **Timezone Reference:** IST only

Support for macOS or Linux desktop clients may be added in future phases but is **not included** in this version.

3. Requirements Specification

3.1 Functional Requirements

3.1.1 User Authentication

- The employee must log in using the provided username and password.
- The application should validate credentials with the server.
- Successful login should bind the device to the employee session.
- Failed login attempts must be logged locally for audit.

3.1.2 Screenshot Capture

- The system must capture screenshots at random intervals within a configured range.
- Minimum and maximum screenshot intervals should be configurable (e.g., 3–10 minutes).
- Screenshots must be stored in encrypted format.
- Each screenshot must include metadata: timestamp (IST), device ID, employee ID, screenshot ID.

3.1.3 Idle Time Tracking

- The system should track keyboard and mouse inactivity.
- Idle threshold should be configurable (e.g., 60 seconds).
- Idle and active periods must be logged accurately.
- Idle logs must be encrypted before local storage.

3.1.4 Local Encrypted Storage

- All captured screenshots and logs must be encrypted using AES.
- Local folder path must be configurable.
- If the local disk becomes full, the app must notify the user or attempt an automatic cleanup based on oldest-first deletion (optional).

3.1.5 Server Sync & Upload

- The system must upload screenshots and logs to the server:
  - Immediately after capture
  - Or at fixed intervals (1 hour by default)
- Upload failures must be retried with exponential backoff.
- All uploads must be HTTPS-secured.
- Uploaded files should be marked in the local DB to avoid duplicates.

3.1.6 Shift-Based Tracking

- Login and tracking must respect employee shift timings.
- Tracking should auto-start when the shift begins.
- Tracking should auto-stop when the shift ends.
- If the employee logs out before shift end, tracking stops.

3.1.7 System Tray Feature

- Application must minimize to the system tray.
- A tray icon should display the app status.
- Right-click tray menu should include:
  - Open Dashboard
  - View Logs
  - Settings
  - Exit (Admin-protected if necessary)

3.1.8 Application Dashboard

- Should display:
  - Current status (tracking/idle/off-shift)
  - Next screenshot timer
  - Last upload status
  - Shift timing
  - Employee profile info
- Dashboard must be simple and lightweight.

3.1.9 Logging

- Application must maintain logs for:
  - Login attempts
  - Screenshot captures
  - Idle/active status events
  - Upload success/failure
  - Errors and exceptions

3.1.10 Auto-Update (Optional for v1)

- The application may support auto-update if included in roadmap.
- Updates must be downloaded securely and installed silently or via prompt.

3.2 Non-Functional Requirements

3.2.1 Performance

- The app must consume minimal CPU and RAM.
- Screenshot capture must not disrupt user work.
- Uploads must be optimized for low bandwidth usage.

3.2.2 Security

- AES encryption for local storage.
- HTTPS for all server communication.
- Authentication tokens must be securely stored.
- No keystrokes, audio, video, or continuous recordings.

3.2.3 Reliability

- The system must run for long hours without crashing.
- Retry logic must handle intermittent connectivity.
- App must automatically restart tracking after unexpected shutdown.

3.2.4 Usability

- Simple and intuitive UI.
- Minimal configuration required from the employee.
- Should not interfere with daily tasks.

3.2.5 Compatibility

- Fully compatible with Windows 10 and Windows 11 (64-bit).
- Must run under standard enterprise user permissions.

3.2.6 Scalability

- Server-side architecture must support thousands of active clients.
- Client uploads should be optimized to handle peak hours.

3.3 User Roles & Permissions

Employee (Standard User)

- Can log in and start working.
- Can view their dashboard.

- Cannot disable tracking during active shift.
- Cannot delete or alter logs.
- Limited access to settings.

Admin (Management)

- May access additional options like:
  - Forced logout
  - Clear local cache
  - Change configuration (intervals, upload times, etc.)

Server-Side Users (HR/Management Dashboard)

- Not part of desktop app, but relevant to system:
  - View employee logs/screenshots
  - View idle/active reports
  - Manage shifts
  - Manage users

3.4 Constraints & Assumptions

Constraints

- Only Windows OS supported in v1.
- IST is the default and only timezone.
- No mobile app support.
- No keystroke logging or audio/video capture.
- Requires stable internet for periodic uploads.

Assumptions

- Employees work within predefined shifts.
- Machines run Windows 10/11 with admin permission available during installation.
- Employees will remain logged into the system during shift unless break or logout event occurs.
- Server APIs and backend will be ready to accept uploads.

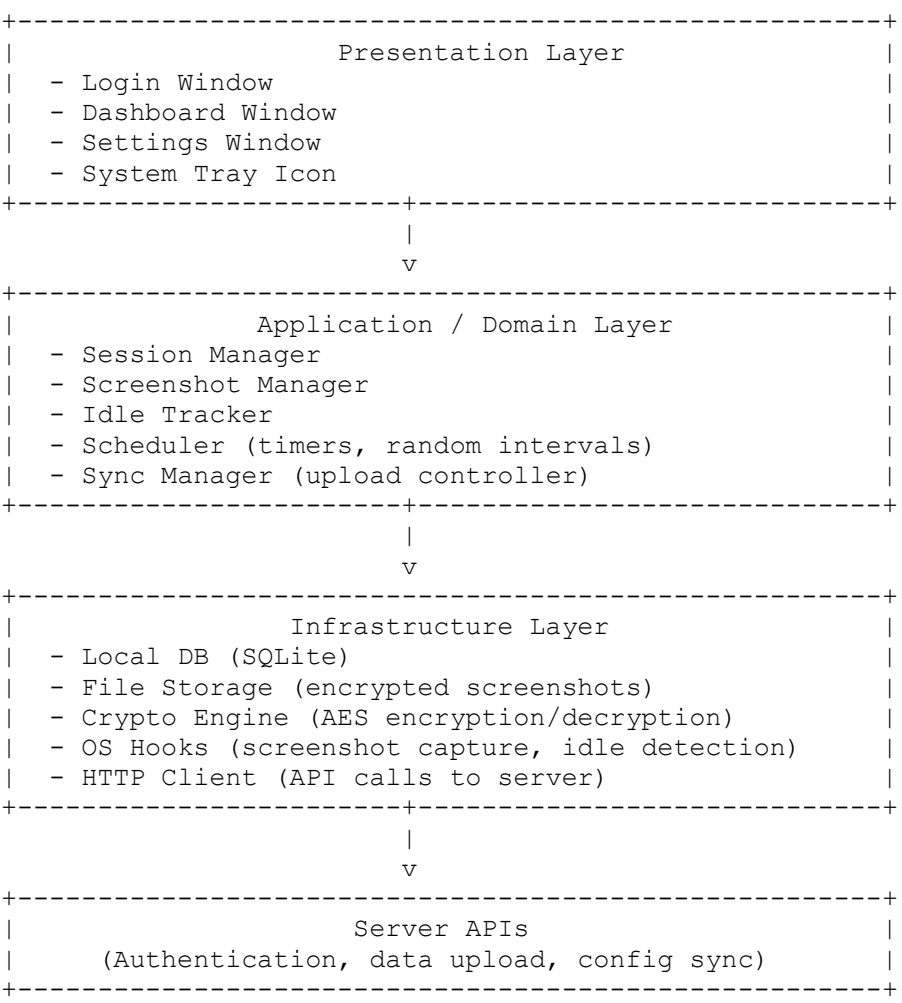
4. Architecture & Design

4.1 System Architecture Overview

ETS follows a modular, layered architecture to keep concerns clean and maintenance easy. At a high level, the system is split into:

- **Presentation Layer (GUI + Tray App)**
- **Application / Domain Layer (business logic)**
- **Infrastructure Layer (storage, encryption, OS integration, HTTP client)**
- **Server Layer (external, not part of this desktop spec, but interacts via APIs)**

High-level view:



The desktop client is responsible for tracking, encrypting, and syncing; the server handles storage, reporting, and management.

4.2 Overall Design Approach

Key design principles:

- **Separation of Concerns:** GUI logic is separated from tracking logic, encryption, and I/O.
- **Event-driven & Timer-based:** Screenshot capture and idle checks run on timers and signals, not blocking the UI.
- **Local-first:** Data is always written locally first (encrypted), then uploaded. If upload fails, data remains queued.
- **Secure-by-default:** Encryption, HTTPS, and minimal data collection are enforced.

Python desktop approach:

- **GUI Framework:**
  - Recommended: **PySide6** (or PyQt5) for modern, native-feeling Windows GUI and system tray support.
- **Background Processing:**
  - Uses Python threads or QTimers/QThreads to avoid freezing the UI.
- **Persistence:**

- SQLite for logs and metadata, filesystem for encrypted screenshot files.

4.3 Module-Level Architecture

4.3.1 Presentation Layer Modules

1. **LoginWindow**
   - Fields: username, password.
   - Actions: login, remember-me (optional).
   - Calls `AuthService` to authenticate.
   - On success: initializes `SessionManager` and opens `DashboardWindow`.
2. **DashboardWindow**
   - Shows:
     - Current tracking status (Active / Idle / Off-shift).
     - Next screenshot (approx. time).
     - Last upload time and status.
     - Shift information (start/end).
   - Buttons:
     - Open Settings
     - View Logs (optional)
   - Read-only for employees (no way to disable tracking).
3. **SettingsWindow**
   - Displays relevant, limited info:
     - Screenshot interval range (read-only for employees, editable for admin if allowed).
     - Upload frequency.
     - Local storage path (read-only for employees).
   - May be locked behind an admin PIN or configuration flag.
4. **SystemTrayIcon**
   - Shows ETS icon with status color (e.g., normal, error).
   - Context menu:
     - Open Dashboard
     - View Logs (optional)
     - Settings (admin)
     - Exit (optionally admin-protected)
   - Reflects current tracking state via icon overlay or tooltip.

4.3.2 Application / Domain Layer Modules

1. **SessionManager**
   - Holds current employee/session info:
     - employee_id, auth_token, shift_times, device_id.
   - Responsible for:
     - Starting/stopping tracking when shift starts/ends.
     - Communicating with `ScreenshotManager`, `IdleTracker`, and `SyncManager`.
2. **ScreenshotManager**
   - Uses OS APIs to capture full screen at random intervals:
     - Random delay between `min_interval` and `max_interval` (e.g., 3–10 minutes).
   - Steps:
     - Generate random next capture time.

        - Take screenshot.
        - Pass raw image to `CryptoEngine` to encrypt and store via `StorageManager`.
        - Create metadata entry in DB.
        - Notify `SyncManager` to queue for upload.
3. **IdleTracker**
    - Periodically checks keyboard/mouse activity using OS hooks.
    - Maintains:
        - `last_input_time`
        - `is_idle` flag.
    - When idle threshold (e.g., 60 seconds) is exceeded:
        - Logs idle start event.
    - When user resumes:
        - Logs idle end event.
    - Writes activity logs to the local DB (encrypted fields where needed).
4. **SyncManager**
    - Maintains a queue of "pending upload" items (screenshots and logs).
    - Runs on:
        - Interval timer (e.g., every 60 minutes).
        - Or immediately after a new screenshot (if configured).
    - Handles:
        - Upload via `HttpClient`.
        - Mark items as "uploaded" in local DB.
        - Retry with backoff for failed uploads.
    - Respects bandwidth and avoids hogging network.
5. **Scheduler / TimerService**
    - Centralizes timer-based events:
        - Random screenshot scheduling.
        - Periodic idle check (e.g., every 5 seconds).
        - Periodic sync.
    - Could be implemented using QTimers (if using PySide/PyQt) or threading timers.

4.3.3 Infrastructure Layer Modules

1. **StorageManager**
    - **SQLite DB** schema (high-level):
        - `employees` (local mapping if needed).
        - `sessions` (login history).
        - `screenshots`:
            - `id`, `employee_id`, `session_id`, `timestamp_ist`, `file_path_encrypted`, `uploaded_flag`.
        - `idle_logs`:
            - `id`, `employee_id`, `session_id`, `idle_start_ist`, `idle_end_ist`, `duration_seconds`, `uploaded_flag`.
        - `app_logs`:
            - `id`, `timestamp_ist`, `level`, `message`, `context`.
    - Provides CRUD-style methods for the domain layer.
2. **FileStorage**
    - Stores encrypted screenshot files on disk.
    - Paths like:
        - `ETS_DATA/YYYY/MM/DD/employee_id/session_id/<screenshot_id>.bin`
    - Ensures directories are created and handled safely.
3. **CryptoEngine**

- Uses Python's `cryptography` library (e.g., AES-256-GCM).
- Responsibilities:
  - Encrypt/decrypt screenshot bytes.
  - Encrypt sensitive fields in DB (tokens, etc.).
- Encryption key:
  - Stored securely, either:
    - Derived from a device key and server key.
    - Or provisioned at install time and stored in protected storage (e.g., Windows Credential Manager) if implemented.

4. **HttpClient**
   - Wraps `requests` or `httpx` library.
   - Handles:
     - Authentication (bearer token, etc.).
     - Upload APIs:
       - `/auth/login`
       - `/upload/screenshot`
       - `/upload/idle-log`
       - `/config/sync` (if needed).
     - Error handling and retries.
     - HTTPS-only communication.
5. **OSIntegration**
   - Platform-specific utilities for Windows:
     - Screenshot capture:
       - Using libraries like `Pillow` + `pyautogui` or native APIs.
     - Idle detection:
       - Using `ctypes` to call `GetLastInputInfo` or similar Windows API.
     - System tray integration:
       - Via PySide6/PyQt features.

4.4 Data Flow Diagrams (Textual)

4.4.1 Screenshot Capture & Upload Flow

```
[TimerService]
   -> triggers random delay
   -> [ScreenshotManager]

[ScreenshotManager]
   -> calls OSIntegration.capture_screen()
   -> raw_image_bytes
   -> CryptoEngine.encrypt_image(raw_image_bytes)
   -> encrypted_bytes
   -> FileStorage.save(encrypted_bytes) => file_path
   -> StorageManager.insert_screenshot_metadata(employee_id, session_id, timestamp_ist,
file_path)
   -> SyncManager.queue_item(screenshot_id)
   -> (optional) notify Dashboard UI
```

Upload:

```
[SyncManager] (on interval or event)
   -> fetch pending screenshots from StorageManager
   -> for each:
        HttpClient.upload_screenshot(metadata + encrypted_file)
        if success:
            StorageManager.mark_screenshot_uploaded(id)
        else:
            schedule retry with backoff
```

4.4.2 Idle Tracking Flow

```
[TimerService] (every X seconds)
   -> IdleTracker.check_idle()

[IdleTracker]
   -> OSIntegration.get_last_input_time()
   -> if now - last_input_time > idle_threshold and not is_idle:
          is_idle = True
          StorageManager.log_idle_start(...)
      elif now - last_input_time <= idle_threshold and is_idle:
          is_idle = False
          StorageManager.log_idle_end(...)
          SyncManager.queue_item(idle_log_id)
```

4.5 Technology Stack

4.5.1 Programming Language

- **Python 3.x** (preferably 3.10+)

4.5.2 GUI

- **PySide6** (recommended) or **PyQt5**

4.5.3 Local Database

- **SQLite**
  - Bundled with Python.
  - No external service needed.
  - Fits desktop use perfectly.

4.5.4 Encryption

- Python `cryptography` library:
  - AES-256-GCM or AES-256-CBC (GCM preferred for authenticated encryption).

4.5.5 HTTP Communication

- `requests` or `httpx` library with HTTPS.

4.5.6 OS Integration

- Screenshot:
  - `Pillow`, `pyautogui`, or direct Windows APIs via `ctypes`.
- Idle detection:
  - Windows API: `GetLastInputInfo`.

4.6 Security Architecture

1. **Encryption at Rest**
   - All screenshots are saved as encrypted binary files.
   - Sensitive tokens and possibly idle logs can be encrypted in DB.
   - Keys:
     - A master key per device, not stored in plain text.
     - Ideally fetched/derived on first login and stored securely.
2. **Encryption in Transit**
   - All communication uses HTTPS.
   - No plain HTTP endpoints allowed.
3. **Authentication**
   - Login via server using username/password.
   - Server returns a short-lived access token and possibly a refresh token.
   - Tokens stored in memory and/or securely in DB (encrypted).
4. **Data Minimization**
   - Only screenshots + activity logs + minimal device info.
   - No keystrokes, audio, or video.
   - Configurable resolution/quality if needed to reduce sensitivity.
5. **Access Control**
   - Employees cannot disable or tamper with logs from UI.
   - Exit or critical settings might require admin-level credentials or be disabled by policy.

4.7 Error Handling & Logging

1. **Error Categories**
   - Network errors (timeouts, DNS, server down).
   - Storage errors (disk full, permission issues).
   - Encryption errors.
   - OS integration errors (screenshot failure, API failures).
2. **Handling Strategy**
   - User-friendly messages in Dashboard for major issues.
   - Automatic retries where possible (network, server).
   - Failsafe:
     - If upload fails, data remains local and queued.
     - If screenshot capture fails, log error and continue scheduling next one.
3. **Logging**
   - Logs written to:
     - Local SQLite table `app_logs`.
   - Log levels:
     - DEBUG (optional), INFO, WARNING, ERROR.
   - For critical errors:

        - Visible indicator in tray icon and Dashboard.
    - Logs may be periodically uploaded to server for diagnostic purposes (if allowed by policy).

5. User Interface (UI/UX) Design

5.1 Design Principles

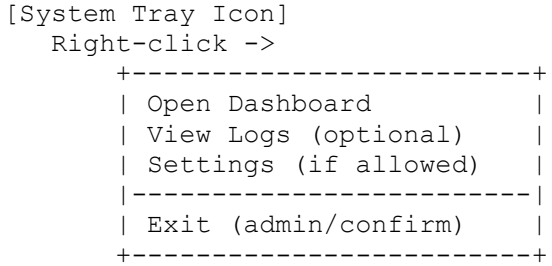
The ETS desktop client UI is designed to be:

- **Simple and distraction-free**
  Only essential information is shown to the employee. No clutter, no complex controls.
- **Non-intrusive**
  The app runs mostly from the system tray. Windows are small, clean, and easy to close/minimize.
- **Consistent**
  Same typography, spacing, and layout patterns across all screens.
- **Status-focused**
  The dashboard prioritizes current tracking status, shift info, and upload state.
- **Safe for non-technical users**
  Employees should not need technical knowledge to use the app.

5.2 Application Navigation Flow

High-level navigation:

System tray interaction:



5.3 Screen Layouts & ASCII Wireframes

These wireframes are conceptual. The developer will translate them into real PySide/PyQt layouts.

5.3.1 Login Window

Goals:

- Simple login.
- Shows app name and company.
- No extra distractions.

```
+-------------------------------------------------+
|          Employee Tracking System (ETS)         |
|          Amaze Internet Services Pvt. Ltd.      |
+-------------------------------------------------+
|                                                 |
|   [ Icon ]   Welcome to ETS                     |
|              Please log in to start tracking.   |
|                                                 |
|   Username:  [_____________________________]    |
|   Password:  [_____________________________]    |
|                                                 |
|   [ ] Remember me                               |
|                                                 |
|          [   Login   ]   [   Exit   ]           |
|                                                 |
|   Status: [ Ready ]                             |
|   Version: v1.0 (Windows, IST)                  |
|                                                 |
+-------------------------------------------------+
```

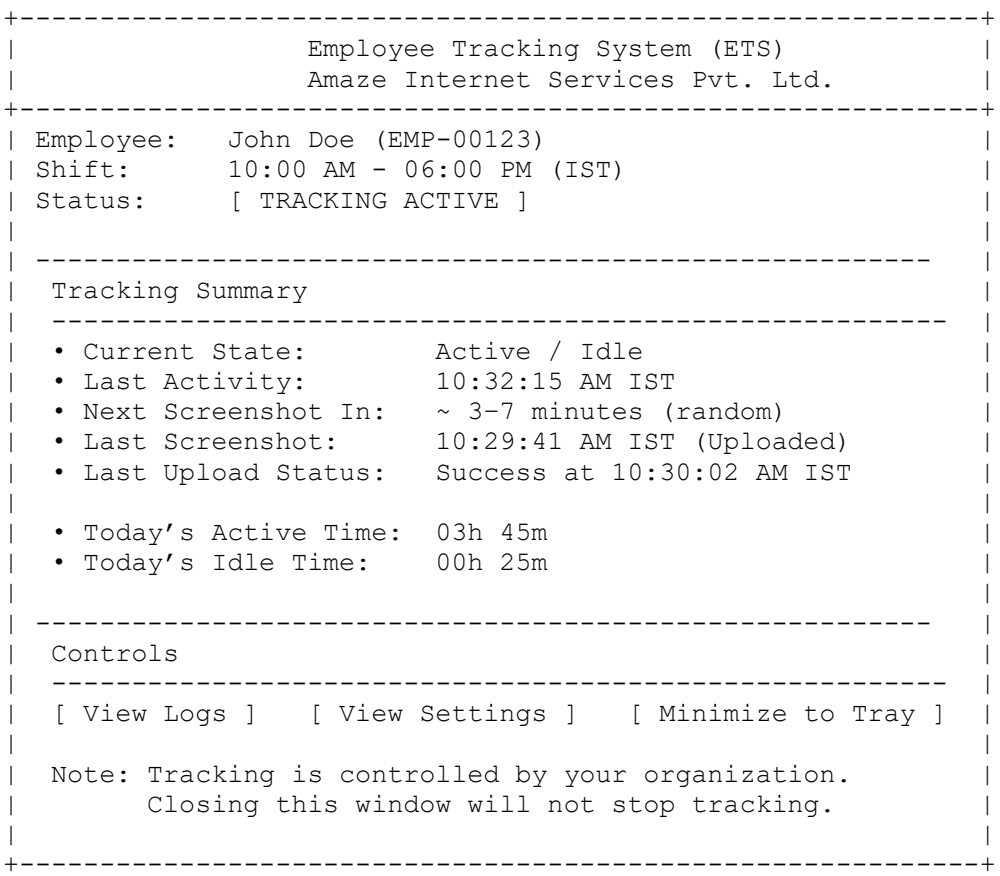
Behavior:

- On **Login**, call the authentication API.
- On success: close this window, open **Dashboard**.
- On error: show a small inline error message above the buttons.

5.3.2 Dashboard Window

Goals:

- Give clear tracking status.
- Show current shift, next screenshot, last upload, and basic stats.
- No ability for employee to disable tracking.

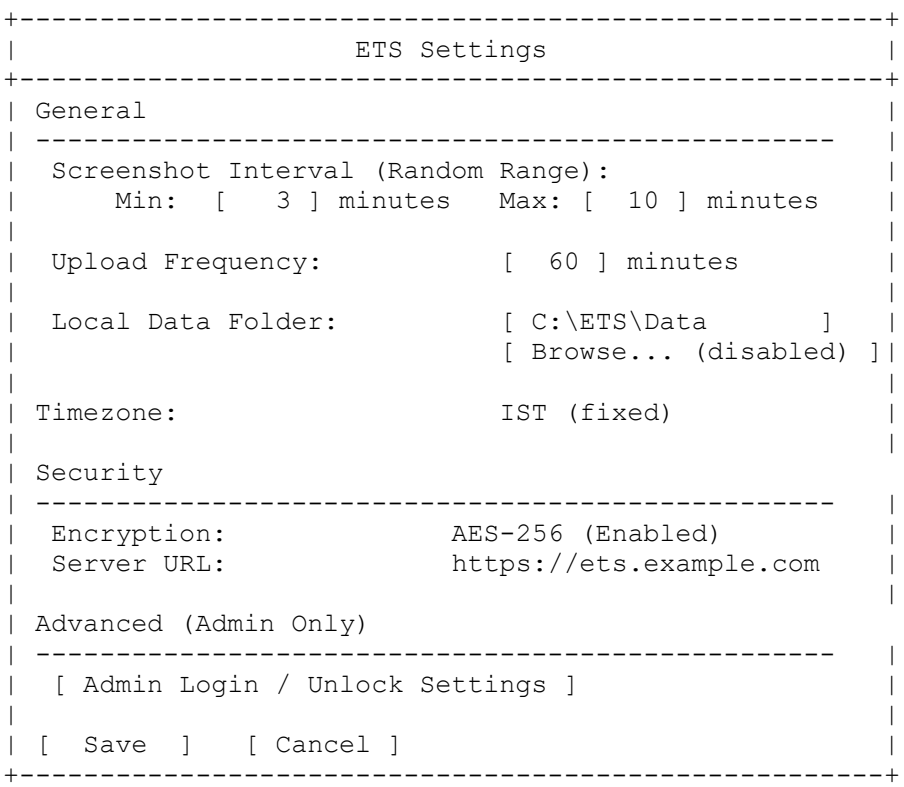


Behavior:

- **Minimize to Tray** hides the window but keeps ETS running.
- **View Logs** opens Logs Window (read-only for employees).
- **View Settings** shows limited settings (most likely read-only or controlled by admin).

5.3.3 Settings Window

Goal:
Allow viewing basic configuration. Any sensitive/critical fields are either read-only or only editable under admin access.

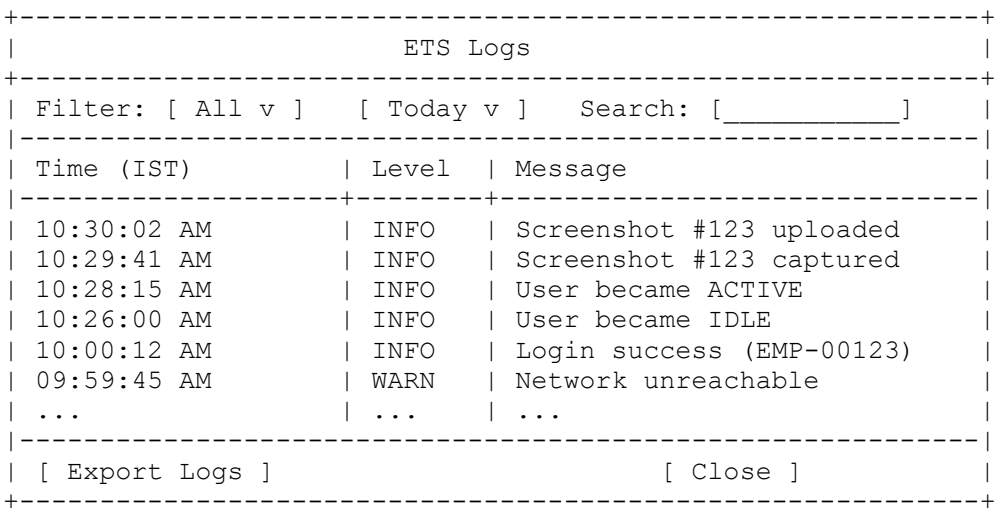


Behavior:

- For normal employees, fields may be read-only and “Save” may be disabled.
- Admin unlock could be done via a separate password/PIN or policy.

5.3.4 Logs Window (Optional but Recommended)

Goal:
Allow user or support to view local app events (read-only).



Export can simply write a text/CSV file for support.

5.3.5 System Tray Menu

Conceptual layout when right-clicking the tray icon:

```
+----------------------------+
|  Employee Tracking System  |
|----------------------------|
|  Open Dashboard            |
|  View Logs                 |
|  Settings                  |
|----------------------------|
|  Exit                      |
+----------------------------+
```

Behavior:

- **Open Dashboard**: restores Dashboard window.
- **View Logs**: opens Logs window.
- **Settings**: opens Settings window (may require admin).
- **Exit**: either:
    - Shows confirmation: "Are you sure? Tracking will stop."
    - Or requires admin credentials (depending on policy).

5.4 Color Scheme & Theme

Suggested default theme (for developer reference):

- **Primary background:** Light neutral (e.g., #F5F5F5 or standard Windows panel color)
- **Text:** Dark gray or black (e.g., #222222)
- **Highlight / Accent:** Soft blue (for buttons and active states)
- **Status colors:**
    - Tracking Active: Green badge/text (e.g., #2E7D32)
    - Idle: Amber (e.g., #FFB300)
    - Error / Upload Failed: Red (e.g., #C62828)

Guidelines:

- Use standard Windows controls where possible to blend with OS.
- Avoid flashy colors; keep it subtle and professional.

5.5 Usability Requirements

- UI must be responsive and should not freeze when screenshots or uploads are happening.
- Windows should remember their last size and position (optional but nice).
- Error messages must be clear and explain:
    - What went wrong.
    - Whether the user needs to do anything.

- Employees should never be confused about whether tracking is on:
  - Dashboard status label.
  - Tray icon tooltip ("ETS: Tracking Active / Idle / Off-shift").
- Keyboard shortcuts:
  - `Esc` to close dialogs where appropriate.
  - `Alt + F4` to close windows (standard Windows behavior).

6. Database & Storage Design

6.1 Data Model Overview

ETS uses a **local SQLite database** plus an **encrypted file storage directory**.

- **SQLite DB** is used for:
  - Sessions
  - Screenshots metadata
  - Idle logs
  - Application logs
  - Configuration (optional)
- **File storage** is used for:
  - Encrypted screenshot files (binary)

Key ideas:

- Every screenshot has:
  - A DB row (metadata)
  - An encrypted file on disk
- Idle periods are stored as start/end timestamps and duration.
- All timestamps are stored in **IST** as `TEXT` (ISO 8601) for simplicity.

6.2 Entity–Relationship Overview (Textual)

Entities and relationships:

```
[employees] 1 --- * [sessions] 1 --- * [screenshots]
                              |
                              +--- * [idle_logs]

[app_logs] is independent (global log table).
[config] (optional) is a key-value store for app settings.
```

- One employee can have multiple sessions (e.g., different days).
- One session can have many screenshots and many idle log entries.

6.3 Table Descriptions

6.3.1 employees

Local reference to server-side employees. May be populated at login.

Field	Type	Description
id	INTEGER	Primary key (local auto-increment)
employee_id	TEXT	Employee ID from server (e.g., EMP-00123)
full_name	TEXT	Full name of employee
email	TEXT	Employee email (optional)
is_active	INTEGER	1 = active, 0 = inactive
created_at_ist	TEXT	ISO datetime in IST

Constraints & Notes:

- `id` is `PRIMARY KEY AUTOINCREMENT`.
- `employee_id` should be `UNIQUE`.
- Index recommended on `employee_id`.

6.3.2 sessions

Represents a login session on the desktop app.

Field	Type	Description
id	INTEGER	Primary key
employee_id	TEXT	FK → employees.employee_id
session_token	TEXT	Encrypted auth token or session identifier
device_id	TEXT	Unique device identifier (generated per machine)
login_time_ist	TEXT	Login timestamp (IST, ISO)
logout_time_ist	TEXT	Logout timestamp (IST, ISO, nullable)
shift_start_ist	TEXT	Shift start time (IST)
shift_end_ist	TEXT	Shift end time (IST)
status	TEXT	e.g., 'ACTIVE', 'COMPLETED', 'INTERRUPTED'

Constraints & Notes:

- Index on `(employee_id, login_time_ist)`.
- `session_token` is stored **encrypted**.

6.3.3 screenshots

Metadata for each screenshot. The actual image is stored separately on disk.

Field	Type	Description
id	INTEGER	Primary key
employee_id	TEXT	FK → employees.employee_id
session_id	INTEGER	FK → sessions.id
timestamp_ist	TEXT	Time screenshot was taken (IST, ISO)

Field	Type	Description
file_path	TEXT	Relative path to encrypted screenshot file
file_size_bytes	INTEGER	Size of the encrypted file
upload_status	TEXT	'PENDING', 'UPLOADED', 'FAILED'
upload_attempts	INTEGER	Number of upload attempts
last_upload_ist	TEXT	Last upload attempt time (IST, nullable)
created_at_ist	TEXT	Record creation time (IST)

Constraints & Notes:

- Index on `(employee_id, session_id, timestamp_ist)`.
- `file_path` is relative to a base data directory.
- The image content is **not** in the DB, only path & metadata.

6.3.4 idle_logs

Stores idle and active periods for a session.

Field	Type	Description
id	INTEGER	Primary key
employee_id	TEXT	FK → employees.employee_id
session_id	INTEGER	FK → sessions.id
idle_start_ist	TEXT	Time when user became idle
idle_end_ist	TEXT	Time when user became active again
duration_seconds	INTEGER	Total idle duration in seconds
upload_status	TEXT	'PENDING', 'UPLOADED', 'FAILED'
upload_attempts	INTEGER	Number of upload attempts
last_upload_ist	TEXT	Last upload attempt time (IST, nullable)
created_at_ist	TEXT	Record creation time (IST)

Constraints & Notes:

- Index on `(employee_id, session_id, idle_start_ist)`.
- For ongoing idle periods, `idle_end_ist` may be null until resolved.

6.3.5 app_logs

Application-level logs for support and debugging.

Field	Type	Description
id	INTEGER	Primary key
timestamp_ist	TEXT	Log time (IST)
level	TEXT	'DEBUG', 'INFO', 'WARN', 'ERROR'
source	TEXT	Module name (e.g., 'ScreenshotManager')

Field	Type	Description
message	TEXT	Log message
context_json	TEXT	Optional JSON string with extra context

Constraints & Notes:

- Index on `(timestamp_ist, level)`.
- Can be periodically truncated / pruned to limit DB size.

6.3.6 config (optional)

Simple key-value store for local configuration.

Field	Type	Description
key	TEXT	Configuration key (unique)
value	TEXT	Configuration value (string)
updated_at	TEXT	Last updated time (IST, ISO)

Examples:

- `screenshot_min_minutes` → `"3"`
- `screenshot_max_minutes` → `"10"`
- `upload_interval_minutes` → `"60"`
- `server_url` → `"https://ets.example.com"`

6.4 Local Storage Structure (Screenshots & DB)

6.4.1 Base Directory

By default (example):

```
C:\ETS\
    ├── data\
    │     └── ets.db          (SQLite database file)
    ├── logs\                 (optional text/rotating log files)
    └── screenshots\
          └── YYYY\
                └── MM\
                      └── DD\
                            └── EMP-00123\
                                  └── SESSION-45\
                                        ├── sc_000001.bin
                                        ├── sc_000002.bin
                                        └── ...
```

- `ets.db` contains all the tables described above.
- Screenshot files are stored as `.bin` (encrypted binary).
- Directory structure:

```
screenshots/YYYY/MM/DD/<employee_id>/<session_id>/<screenshot_id>.bin
```

Note:
`session_id` can be represented either as the numeric DB `id` or a formatted identifier (`SESSION-45`), as long as it's consistent.

6.4.2 File Naming Convention

For each screenshot:

- Suggested filename: `sc_<screenshot_db_id>.bin`
  Example: `sc_12345.bin`

This avoids generating separate GUIDs and keeps it trivial to map DB rows to files.

6.4.3 Cleanup Strategy (High Level)

To control disk usage:

- A background job can:
  - Check total size of `screenshots` directory.
  - Delete oldest screenshots **only if**:
    - They are already uploaded (`upload_status = 'UPLOADED'`).
- Optionally, enforce:
  - Max days to retain screenshots locally (e.g., 30 days).

Actual cleanup rules can be defined in a later section or config.

6.5 Encryption & Data Protection

6.5.1 Encryption Algorithm

- **Algorithm:** AES-256 (GCM preferred for authenticated encryption)
- **Library:** Python `cryptography` (e.g., `Fernet` / `AESGCM` implementations)
- **Data encrypted:**
  - Screenshot binary content (files)
  - Sensitive tokens/session secrets in the DB
  - Optionally: email, device_id, etc., if policy requires

6.5.2 Key Management (Client-Side)

High-level approach (to be refined in implementation):

- A **device key** is generated and stored securely on first install.
- A **server-assigned key** or seed can optionally be used to derive the final key.
- The final AES key is never stored in plain text in the DB or filesystem.

Possible key storage options:

- Windows-specific:
  - Use **Windows Credential Manager** or DPAPI for key storage.
- Simple v1 approach:
  - Encrypt the AES key with a machine-bound key and store it in a small file under `C:\ETS\keys\`.

6.5.3 Encryption Flow (Screenshots)

```
[Raw Screenshot Bytes]
       |
       v
[CryptoEngine.encrypt_image()]
       |
       v
[Encrypted Bytes]
       |
       v
[FileStorage.save(encrypted_bytes)]
       |
       v
[file_path recorded in DB -> screenshots.file_path]
```

Decryption (if needed on client):

```
[FileStorage.read(file_path)]
       |
       v
[CryptoEngine.decrypt_image(encrypted_bytes)]
       |
       v
[Decrypted Image Bytes -> Display or Upload]
```

Note:
If the server accepts the encrypted bytes as-is and decrypts on server-side, the client may not need to decrypt for upload. In that case, upload uses the same encrypted file content.

6.5.4 Protection of Sensitive Information

- **Session tokens** in `sessions.session_token`:
  - Stored only after encryption.
- Logs (`app_logs`) must **not** contain raw sensitive data (e.g., full tokens, passwords).
- Passwords are **never** stored locally:
  - Used once for login, then discarded.

7. API Specification

The ETS desktop client communicates with a central server using a secure **HTTPS-based REST API**. All responses are in **JSON format** unless specified otherwise. All timestamps exchanged with the server should use **IST** or ISO 8601 format with IST offset.

All APIs must:

- Use **HTTPS only**
- Return clear status codes (200, 400, 401, 403, 500)

- Require **Authorization: Bearer <token>** header for authenticated routes
- Be optimized for low-bandwidth clients

7.1 Purpose of APIs

The ETS client requires the backend for:

1. User authentication
2. Receiving shift timing and configuration
3. Uploading screenshots
4. Uploading idle and activity logs
5. Checking server-status, updates, or configuration changes
6. Secure logout and session termination

The backend may be implemented in Node/PHP/Python/Go—this document only defines the protocol.

7.2 Authentication Flow

POST /auth/login

Description:
Authenticates the employee and starts a new session.

Headers:
```
Content-Type: application/json
```

Request Body:

```
{
  "employee_id": "EMP00123",
  "password": "user_password",
  "device_id": "WIN-DEVICE-XYZ"
}
```

Response (Success):

```
{
  "status": "success",
  "token": "<JWT OR ACCESS TOKEN>",
  "employee": {
    "employee_id": "EMP00123",
    "full_name": "John Doe"
  },
  "shift": {
    "start_ist": "2025-03-15T10:00:00+05:30",
    "end_ist": "2025-03-15T18:00:00+05:30"
  },
  "config": {
    "screenshot_min_minutes": 3,
    "screenshot_max_minutes": 10,
    "upload_interval_minutes": 60
  }
}
```

Response (Failure):

```
{
  "status": "failed",
  "message": "Invalid credentials."
}
```

POST /auth/logout

Description:
Ends the session for the user.

Headers:

- `Authorization: Bearer <token>`
- `Content-Type: application/json`

Body:

```
{
  "device_id": "WIN-DEVICE-XYZ"
}
```

Response:

```
{
  "status": "success",
  "message": "Session closed."
}
```

7.3 Screenshot Upload API

POST /upload/screenshot

Description:
Uploads an **encrypted** screenshot to the server.

Headers:

```
Authorization: Bearer <token>
Content-Type: multipart/form-data
```

Fields:

- `employee_id`: string
- `session_id`: string or numeric
- `timestamp_ist`: ISO timestamp
- `file`: encrypted binary screenshot file
- `file_size`: number (bytes)

Example Request (Form Data):

```
employee_id = EMP00123
session_id = 45
timestamp_ist = 2025-03-15T10:29:41+05:30
file = <binary file>
file_size = 247890
```

Response:

```
{
  "status": "success",
  "screenshot_id": 12345
}
```

Failure Response:

```
{
  "status": "failed",
  "message": "File too large or corrupted."
}
```

7.4 Idle Log Upload API

POST /upload/idle-log

Description:
Uploads a single idle event record.

Headers:
```
Authorization: Bearer <token>
Content-Type: application/json
```

Request Body:

```
{
  "employee_id": "EMP00123",
  "session_id": 45,
  "idle_start_ist": "2025-03-15T12:05:00+05:30",
  "idle_end_ist": "2025-03-15T12:12:05+05:30",
  "duration_seconds": 425
}
```

Response:

```
{
  "status": "success",
  "idle_log_id": 981
}
```

7.5 Config Sync API

POST /config/sync

Description:
Used by the client to sync updated settings, screenshots interval, forced logout notices, or shift changes.

Headers:
```
Authorization: Bearer <token>
Content-Type: application/json
```

Request Body:

```
{
  "employee_id": "EMP00123",
  "device_id": "WIN-DEVICE-XYZ"
}
```

Response:

```
{
  "status": "success",
  "config": {
    "screenshot_min_minutes": 3,
    "screenshot_max_minutes": 12,
    "upload_interval_minutes": 30,
    "force_logout": false,
    "shift": {
      "start_ist": "2025-03-15T10:00:00+05:30",
      "end_ist": "2025-03-15T18:00:00+05:30"
    }
  }
}
```

If server requires immediate logout:

```
{
  "status": "success",
  "config": {
    "force_logout": true
  }
}
```

7.6 Health Check API

GET /status/ping

Description:
Used to confirm that the API server is reachable.

Headers:
None required.

Response:

```
{
  "status": "ok",
  "server_time_ist": "2025-03-15T11:22:00+05:30"
}
```

7.7 Error Codes & Responses

HTTP Code	Meaning	Example
200	Success	`{ "status": "success" }`
400	Bad Request	`{ "message": "Invalid format" }`
401	Unauthorized	`{ "message": "Invalid or expired token" }`
403	Forbidden	`{ "message": "Access denied" }`
404	Not Found	`{ "message": "Endpoint not found" }`
500	Internal Server Error	`{ "message": "Unexpected server error" }`

7.8 Security Requirements

- Token must be **revocable** by server.
- Token must expire (short-lived token recommended).
- OAuth2/JWT or custom token system supported.
- All file uploads must be validated server-side.
- Device ID must be unique and consistent across launches.

8. Development Plan

8.1 Milestones

The development of the Employee Tracking System (ETS) desktop application will be executed in phased milestones to ensure clarity, control, and predictable delivery.

Milestone 1 – Project Setup & Core Framework

- Define final tech stack (Python, GUI framework, libraries).
- Initialize repository structure.
- Implement basic application skeleton with a simple window and system tray icon.
- Configure logging and configuration management.
- Basic health-check screen to confirm app runs on Windows 10/11.

Milestone 2 – Authentication & Session Management

- Implement Login window UI.
- Integrate `/auth/login` and `/auth/logout` APIs.
- Implement `SessionManager` with local session persistence.
- Store employee info locally in SQLite.
- Basic error handling for login failures and API connectivity issues.

Milestone 3 – Screenshot Capture & Idle Tracking

- Implement `ScreenshotManager` with random interval scheduling.
- Implement `IdleTracker` using Windows idle detection.
- Store screenshot metadata and idle logs in SQLite.
- Implement encryption and file storage for screenshots.
- Basic dashboard to display current status and last events.

Milestone 4 – Sync & Background Operations

- Implement `SyncManager` for queued upload of screenshots and idle logs.
- Integrate `/upload/screenshot` and `/upload/idle-log` APIs.
- Implement retry logic for failed uploads.
- Add `/config/sync` integration for server-driven settings and shifts.
- Improve tray icon behavior and status indication.

Milestone 5 – UI Polish, Settings & Logs

- Complete Dashboard UI according to wireframes.
- Implement Settings window (read-only or admin-locked options).
- Implement Logs window (view app logs, filter by date/level).
- Refine usability (messages, tooltips, shortcuts).
- Add basic export of logs for support.

Milestone 6 – Testing, Optimization & Release

- Functional testing of all major flows.
- Performance and resource usage checks (CPU, RAM, disk).
- Stability testing for long-running sessions.
- Bug fixing and small UX improvements.
- Build packaging (installer) and internal pilot rollout.

8.2 Sprint Breakdown (Agile, Single Developer)

Assuming 1–2 week sprints and a single primary developer (Aushutosh Kumar), the work can be broken down as follows. This is a suggested plan; durations can be adjusted.

Sprint 1 – Environment & Skeleton

Goals:

- Get a working base app on Windows using Python.
- Ensure logging, config, and basic window/tray are running.

Key Tasks:

- Set up Python virtual environment and dependencies (PySide6/PyQt5, SQLite, cryptography, requests/httpx, etc.).
- Initialize Git repository and branching strategy. (Optional)
- Implement base application entry point.
- Create a minimal main window with:
  - App title
  - Status label
- Implement a basic system tray icon with:
  - “Open” and “Exit” menu options.
- Implement `app_logs` table and logging helper.
- Basic `config` table or config file read/write.

Sprint 2 – Login & Session Layer

Goals:

- Implement login flow and persistent session handling.

Key Tasks:

- Implement `LoginWindow` UI as per wireframe.
- Create `AuthService` to call `/auth/login`.
- Implement `SessionManager`:
  - Store employee info and session token in SQLite (encrypted token).
  - Track login_time, shift timings, device_id.
- Implement `employees` and `sessions` tables with CRUD helpers.
- Error handling for:
  - Invalid credentials.
  - Network errors.
- On successful login:
  - Open Dashboard.
  - Close Login window.

Deliverable: User can log in and see a basic Dashboard with their name and shift info.

Sprint 3 – Dashboard UI, Idle Tracking, & Basic Screenshot Capture

Goals:

- Build Dashboard UI.
- Implement idle detection and initial screenshot capture logic.

Key Tasks:

- Implement `DashboardWindow` layout:
  - Employee name
  - Shift info
  - Status (Active/Idle/Off-Shift)
  - Last activity time
  - Next screenshot estimate
- Implement `IdleTracker` using Windows APIs (`GetLastInputInfo` via `ctypes` or similar).
- Implement `idle_logs` table and logging of idle/active transitions.
- Implement `ScreenshotManager` with:
  - Full screen capture.
  - Random interval scheduling between min/max minutes.
- Implement `screenshots` table and local metadata logging.
- Wire the Dashboard to display:
  - Last screenshot time
  - Today's idle/active time summary (basic approximation).

Deliverable: App can log in, track idle/active state, and take screenshots at random intervals, storing everything locally.

Sprint 4 – Encryption, File Storage & Sync Manager

Goals:

- Make storage secure and start synchronizing data with the server.

Key Tasks:

- Design and implement `CryptoEngine`:
  - AES-256 encryption/decryption for screenshot files.
  - Encryption for sensitive DB fields where required.
- Implement `FileStorage`:
  - Directory structure for screenshots.
  - Functions to save & read encrypted screenshot files.
- Ensure `ScreenshotManager` now encrypts and writes `.bin` files.
- Implement `SyncManager`:
  - Queue pending screenshots and idle_logs.
  - Implement upload loops:
    - `/upload/screenshot`
    - `/upload/idle-log`
  - Handle network errors with retry/backoff.
- Update Dashboard to display last upload status and time.

Deliverable: App securely stores data and can upload logs and screenshots to the server when network is available.

Sprint 5 – Settings, Logs UI & Config Sync

Goals:

- Provide a usable Settings window, Logs view, and config sync.

Key Tasks:

- Implement `SettingsWindow`:
  - Display config values (screenshot intervals, upload frequency, local data folder, server URL, encryption info).
  - Make employee-facing fields read-only.
  - Add admin unlock section (if in scope).
- Implement `LogsWindow`:
  - Display `app_logs` with filter by date/level.
  - Optional search box and “Export Logs” to text/CSV.
- Implement `/config/sync` integration:
  - Add a periodic sync job for updated server config and shift changes.
  - Apply new config values to `ScreenshotManager` and `SyncManager` without restarting the app.
- Enhance System Tray behaviors:
  - Tray tooltips showing “Tracking Active / Idle / Off-shift / Error”.
  - Add “View Logs” and “Settings” entries to tray menu.

Deliverable: App becomes support-friendly and configurable via server-driven parameters.

Sprint 6 – Stabilization, Testing & Packaging

Goals:

- Finish, stabilize, and ship a usable v1 build.

Key Tasks:

- Write and execute functional test cases for:
  - Login/logout behavior.
  - Idle detection.
  - Screenshot capture.
  - Upload flows.
- Test behavior under:
  - Long-running scenarios (full shift).
  - Network loss/restoration.
  - Disk space issues (low space simulation).
- Fix high-priority bugs and refine UX texts.
- Implement installer packaging (e.g., PyInstaller + NSIS/Inno Setup).
- Create internal pilot build.
- Collect feedback and apply minor adjustments.

Deliverable: ETS v1 Windows installer ready for internal rollout.

8.3 Task Allocation

Given the current plan, all engineering work will be handled by a single full-stack developer, while reviews remain with HR and the CEO.

Primary Developer (Full Stack)

Aushutosh Kumar

Responsibilities:

- Full development of the ETS desktop application
  - Python application logic
  - GUI development (PySide/PyQt)
  - Local database implementation (SQLite)
  - Encryption layer (AES-256)
  - Screenshot capture and idle tracking modules
  - File storage system
  - Background sync, retry logic, timers
  - System tray integration
- Full backend development
  - API implementation (login, screenshot upload, idle log upload, config sync, logout)
  - Database design and server-side schema
  - Server authentication and token-based security
  - File handling for uploaded data

    - Server-side logging and API error handling
- Deployment preparation for both client (installer) and backend (server hosting)

Tech Lead

Shailabh Suman

Responsibilities:

- Architecture decisions for client and backend
- Reviewing and validating all technical modules
- Finalizing API specifications and data structures
- Ensuring proper encryption, security, and performance standards
- Technical documentation, versioning, and delivery oversight
- Coordinating feedback loops with the organization

Review & Validation

Sujeet Kumar (HR)
Responsibilities:

- Review functional behavior related to working hours, shift logic, monitoring guidelines
- Validate that the tool meets HR expectations without violating privacy boundaries

Saurabh Suman (CEO)
Responsibilities:

- Final approval before release
- Ensure alignment with company monitoring policy, compliance, and business goals

---

8.4 Development Roadmap (High-Level Timeline)

This is a sample timeline assuming 1-week sprints. It can be adjusted based on priority and availability.

- **Week 1:** Sprint 1 – Setup & Skeleton
- **Week 2:** Sprint 2 – Login & Session
- **Week 3:** Sprint 3 – Dashboard, Idle Tracking, Basic Screenshots
- **Week 4:** Sprint 4 – Encryption, File Storage, Sync
- **Week 5:** Sprint 5 – Settings, Logs, Config Sync
- **Week 6:** Sprint 6 – Testing, Optimization, Packaging, Pilot

After the pilot, a **post-pilot iteration** may be added to address feedback and plan future enhancements (Linux/macOS clients, advanced reporting integrations, etc.).

9. Testing Strategy

The goal of the testing strategy is to ensure that the Employee Tracking System (ETS) is stable, accurate, secure, and reliable under real working conditions. Testing must cover functionality, performance, usability, edge cases, and long-duration behavior since the application is expected to run continuously during work shifts.

9.1 Test Plan Overview

The ETS client will undergo a structured testing plan that includes:

- **Unit Testing** (module-level Python tests)
- **Integration Testing** (API and local DB interactions)
- **Functional Testing** (full workflows)
- **Performance & Load Testing** (long-running reliability)
- **Security Testing** (encryption, authentication, data safety)
- **Installation & Update Testing**
- **User Acceptance Testing (UAT)** with HR and CEO

All tests must be performed on Windows 10 and Windows 11 (64-bit).

9.2 Functional Test Cases

Functional testing ensures the ETS desktop app behaves exactly as intended. Key scenarios include:

9.2.1 Authentication

- Login with valid credentials → success
- Login with invalid credentials → error message
- Login without internet → proper error message
- Server down during login → retry logic, graceful message
- Logout behavior → session ends correctly

9.2.2 Session & Shift Handling

- Tracking starts at login
- Tracking stops at logout
- Tracking auto-starts at shift start time
- Auto-stop at shift end
- Forced logout via `/config/sync`

9.2.3 Screenshot Capture

- Random screenshot intervals within configured range
- Screenshots stored encrypted
- Metadata inserted correctly in DB
- Screenshot stored at correct directory structure
- Failure to capture → app logs error & continues running

9.2.4 Idle Detection

- Idle state recorded after threshold
- Active state recorded on user input return
- Idle duration calculated correctly
- Idle logs stored in DB and uploaded successfully

9.2.5 Sync & Upload

- Successful upload of screenshot files
- Successful upload of idle logs
- Failed upload attempts lead to retries
- Upload queue functions correctly
- Upload must not block UI

9.2.6 System Tray & UI

- Tray icon visibility
- Tray tooltip reflects real-time state
- Tray menu actions work
- Dashboard information updates properly
- Settings window read-only/locked for employees

9.2.7 Logging

- All important events recorded in `app_logs`
- Logs filtered in Logs Window
- Export logs function works

9.3 System & Integration Tests

Integration tests ensure all modules work together.

9.3.1 DB Integration

- SQLite tables create correctly
- Data inserted/retrieved accurately
- Encrypted fields decrypt correctly
- Local DB remains consistent during long runs

9.3.2 File Storage Integration

- Encrypted screenshot files write correctly
- File names and paths match DB metadata
- Corrupted or unreadable files handled gracefully

9.3.3 API Integration

- Login, upload, config sync work end-to-end
- Token validation
- Correct API responses under:
    - Timeouts
    - High latency
    - Server offline
    - Incorrect data formats

9.3.4 Encryption Validation

- Screenshots always encrypted at rest
- Token and sensitive data encrypted in DB
- Decryption process reliable

9.4 Performance Testing

ETS must run continuously without freezing, crashing, or slowing down the system.

9.4.1 CPU & Memory Usage

- Measure CPU usage during:
  - Idle detection loop
  - Screenshot capture
  - Upload cycles
- Check memory growth over long sessions (no leaks)

9.4.2 Long-Duration Stability (Shift Simulation)

Simulate an 8–12 hour continuous shift:

- Screenshots taken randomly
- Idle/active activity recorded
- Upload cycles triggered
- No hangs, crashes, or data loss

9.4.3 Disk Space Testing

- Simulate low disk space
- App gracefully stops saving screenshots
- Display warning message or log the failure

9.5 Installation & Update Testing

9.5.1 Installer Testing

- Installer runs on Windows 10 and 11
- Installs required files to correct directory
- Creates local DB
- Creates folders for data, logs, screenshots
- Optionally adds startup entry (if planned)

9.5.2 First Launch

- App initializes properly
- No crashes due to missing folders or permissions

9.5.3 Update Testing

- Upgrade installer installs new version over old
- Configuration preserved
- Existing stored data remains intact
- Backward compatibility tests for DB schema changes

9.6 Bug Tracking Approach

A simple bug-tracking workflow will be used:

1. Developer logs issues in a tracker (Jira, GitHub Issues, or internal tracker).
2. Each bug includes:
    - Steps to reproduce
    - Expected behavior
    - Actual behavior
    - Logs/screenshots
3. Bugs classified as:
    - Critical
    - High
    - Medium
    - Low
4. Critical and High severity bugs prioritized for immediate fix.
5. Bugs validated by Tech Lead (Shailabh Suman) before closing.

10. Deployment & Distribution

The Employee Tracking System (ETS) is distributed as a Windows desktop application. This section outlines how the application is packaged, installed, updated, and maintained across employee machines.

10.1 Build Process

The ETS application is a Python-based project packaged into a standalone Windows executable. The build process includes:

10.1.1 Build Tools

- **PyInstaller** (recommended)
    - Converts Python scripts into a single EXE or folder-based distribution.
- Optional:
    - **UPX** for executable compression
    - **Inno Setup** or **NSIS** for professional installers

10.1.2 Build Steps

1. Activate the project's virtual environment.
2. Install all required dependencies.
3. Run a PyInstaller build command:

4. `pyinstaller --onefile --noconsole ets_main.py`

   or for a GUI app:

   ```
   pyinstaller --windowed --icon=app.ico ets_main.py
   ```

5. Verify:
   - EXE starts correctly
   - All assets (icons, fonts, config files) included

10.1.3 Output

A typical build output folder will include:

```
ETS/
  ├── ETS.exe
  ├── runtime files
  ├── /assets
  └── /config
```

This folder is then packaged into an installer.

10.2 Installer Packaging

A professional installer ensures consistent deployment across machines.

10.2.1 Installer Framework

Recommended options:

- **Inno Setup** (preferred)
- **NSIS**
- **Advanced Installer** (paid but more powerful)

10.2.2 Installer Requirements

The installer should:

- Copy ETS to `C:\Program Files\ETS\`
- Create necessary folders:
- `C:\ETS\data\`
- `C:\ETS\logs\`
- `C:\ETS\screenshots\`
- Install dependencies used by the application
- Create Start Menu shortcut
- Add option: **"Start ETS automatically when Windows starts"**
- Run ETS on completion (optional)
- Prompt for admin permission if required

10.2.3 Environment Requirements

The installer must check:

- Windows 10 or 11 (64-bit)
- Minimum 2 GB RAM
- At least 500 MB free disk space
- Python is **not** required to be installed on the machine (app is fully packaged)

10.3 System Requirements

Minimum Requirements

- OS: Windows 10 (64-bit)
- CPU: Dual-core
- RAM: 2 GB
- Disk: 500 MB free space
- Internet: Required for uploads and config sync

Recommended Requirements

- OS: Windows 11 (64-bit)
- RAM: 4 GB
- Stable internet connection

10.4 Versioning Strategy

ETS will follow a **semantic versioning** pattern:

```
MAJOR.MINOR.PATCH
```

Examples:

- **1.0.0** → First stable release
- **1.1.0** → New features added (e.g., better dashboard, improved sync)
- **1.1.3** → Bug fixes

What increments mean:

- **MAJOR:** Breaking changes or big new feature sets
- **MINOR:** Backward-compatible new features
- **PATCH:** Bug fixes, performance improvements

The version number will be shown in:

- Login window
- Dashboard (footer)
- Installer “About” section

10.5 Update & Patch Delivery

ETS will support simple update mechanisms.

10.5.1 Manual Update (v1 recommended)

Admins can distribute a new installer via:

- Email
- Internal portal
- Company network share

Employees run the installer, which upgrades files in place.

10.5.2 Silent Update (optional future)

ETS downloads updates automatically from server:

- Downloads new EXE
- Verifies checksum
- Replaces the old version
- Restarts the app

This requires:

- A backend update server
- Update API
- Auto-update service inside the client

Not mandatory for v1.

10.6 Error Handling During Deployment

Installer Failures

If installation fails:

- Log written to `C:\ETS\installer.log`
- Common causes:
  - Insufficient permissions
  - Disk full
  - Corrupt installer

First Launch Failures

ETS must:

- Log error to `app_logs`

- Show a friendly message
- Never crash silently

10.7 Distribution Strategy

ETS can be distributed internally using:

Option A – Direct Download

- Hosted on a secure internal URL
- Employees download installer and run it

Option B – USB / Offline (If necessary)

For restricted networks:

- Installer provided via USB drive
- Works fully offline except login and uploads

Option C – Company IT Deployment (Optional for Future Enhancements)

- IT department pushes installer via:
  - Group Policy
  - SCCM / Intune
  - LAN deployment tool

Security Checks

All installers must be:

- Digitally signed (recommended)
- Virus scanned
- Version controlled

11. User Documentation

This section guides employees and administrators on how to install, use, and troubleshoot the Employee Tracking System (ETS). The goal is to provide clear, practical guidance without technical complexity.

11.1 Quick Start Guide

Step 1: Install the Application

1. Run the ETS installer provided by the company.
2. Follow the instructions on-screen.

3. Once installed, ETS will appear in the Start Menu and may launch automatically.

Step 2: Launch ETS

- Open ETS from the Start Menu or system tray.
- On first launch, ETS may create required folders automatically:
  - `C:\ETS\data\`
  - `C:\ETS\logs\`
  - `C:\ETS\screenshots\`

Step 3: Log In

1. Enter your **Employee ID** and **Password**.
2. Press **Login**.
3. If credentials are valid, you will be taken to the **Dashboard**.
4. If login fails, an error message will appear.

Step 4: Keep ETS Running

- After login, ETS runs quietly in the background.
- Always check the **System Tray** icon to confirm:
  - Green → Tracking Active
  - Amber → Idle
  - Red → Upload Issue
  - Grey → Off Shift / Not Tracking

Step 5: End of Day

- You may close the dashboard window.
- Tracking automatically stops when:
  - You log out from ETS
  - Your shift ends

Note: Closing the dashboard does **not** stop tracking. Only logging out ends tracking.

11.2 Feature Descriptions

11.2.1 Login Window

- Allows access to ETS using valid credentials.
- Shows status of login attempt and version number.

11.2.2 Dashboard

Displays key information:

- Your name and Employee ID
- Shift start and end times
- Current status: Active / Idle / Off-shift
- Last screenshot timestamp
- Next screenshot (approx. time)

- Last upload status
- Daily activity summary

Buttons:

- **View Logs** (optional)
- **Settings** (read-only for employees)
- **Minimize to Tray**

11.2.3 System Tray

The ETS tray icon shows real-time status.

Right-click menu includes:

- Open Dashboard
- View Logs
- Settings
- Exit
  (Exit may require admin approval depending on configuration.)

11.2.4 Idle Tracking

- ETS monitors keyboard and mouse activity.
- If there is no activity for a configured time (example: 60 seconds), ETS marks you as **Idle**.
- When activity resumes, ETS logs the idle end time.

11.2.5 Screenshot Capture

- Screenshots are taken at random intervals (e.g., every 3–10 minutes).
- Screenshots are:
  - Encrypted
  - Saved locally
  - Uploaded automatically to the server

Employees cannot disable screenshots or change the interval.

11.2.6 Data Upload

- ETS uploads:
  - Screenshot files
  - Idle logs
  - Session logs
- Upload happens immediately or on hourly intervals depending on settings.

11.3 Troubleshooting Guide

Unable to Login

Possible causes:

- Incorrect Employee ID or Password
- No Internet connection
- Server temporarily unavailable

What to do:

- Check credentials
- Restart ETS
- Check your Internet connection
- Contact support if the problem persists

ETS Shows Red Icon (Upload Failed)

Meaning:

- ETS could not upload data.

Possible reasons:

- Internet down
- Server not reachable
- Firewall restrictions

What to do:

- Ensure Internet connection is active
- Restart the application
- If still failing, contact IT

Screenshots Not Updating / Dashboard Not Refreshing

Possible reasons:

- Screenshot module error
- Missing permissions
- Display adapter errors

What to do:

- Restart ETS
- Ensure you did not block the app in antivirus
- Check for low disk space
- Logs Window may show detailed errors

ETS Did Not Start Automatically

Check:

- If “Start with Windows” is enabled
- If ETS was blocked by antivirus
- If the installation was incomplete

What to do:

- Reinstall ETS
- Ask IT to check your startup configuration

Idle Status Incorrect

Reasons:

- USB device blocking system input
- Third-party applications preventing correct idle detection
- System issue with Windows last-input API

What to do:

- Unplug unnecessary USB devices
- Restart the PC
- Contact support if persistent

Installer Fails

Common reasons:

- Lack of admin permission
- Disk full
- Corrupted installer file

What to do:

- Run Installer as Administrator
- Free up space
- Re-download the installer

11.4 Frequently Asked Questions (FAQ)

Q1: Can I stop ETS from taking screenshots?
A: No. ETS is controlled by the organization and screenshots are taken automatically during your shift.

Q2: Can ETS see my personal files?
A: ETS only captures screenshots and activity status. It does not access personal files directly.

Q3: Does ETS record audio or keystrokes?
A: No. ETS does **not** record audio, video, keystrokes, or webcam data.

Q4: What happens if Internet disconnects?
A: ETS stores all data locally and uploads automatically when the connection returns.

Q5: What if I shut down the PC?
A: ETS tracking will end. On next login, it will start a new session.

Q6: Does ETS slow down my computer?
A: No. ETS is optimized to use minimal CPU and memory resources.

Q7: Can HR or management see screenshots immediately?
A: Yes, once they are uploaded.

Q8: What if the shift changes?
A: New shift timings are automatically synchronized when `/config/sync` runs.

12. Maintenance & Support

The Maintenance & Support section defines how ETS will be monitored, updated, and maintained after deployment. The goal is to ensure long-term reliability, stability, and compliance with organizational expectations.

12.1 Log Monitoring

ETS generates several categories of logs to help developers and support teams diagnose issues.

12.1.1 Local Log Categories

- **Application Logs (app_logs table)**
  Contains general app events (info, warning, error).
- **Screenshot Logs (screenshots table)**
  Tracks capture time, file path, upload status.
- **Idle Logs (idle_logs table)**
  Records all idle/active transitions.

12.1.2 Log Rotation

To prevent the database from growing too large:

- Retain application logs for **30 days**
- Delete entries older than 30 days automatically
- Screenshot metadata is deleted only when the screenshot file itself is deleted
- Never delete logs that have not been successfully uploaded

12.1.3 Support Usage

Logs help support teams troubleshoot:

- Failed uploads
- API issues
- Screenshot capture issues
- Idle detection failures
- Encryption or file-write issues

Logs must never include sensitive information like passwords or raw tokens.

12.2 Crash Handling

ETS must be resilient and recover gracefully.

12.2.1 Crash Detection

If an unexpected error occurs:

- ETS logs the error as **ERROR** in `app_logs`
- Shows a minimal user-friendly message (if applicable)
- Attempts to restart background services

12.2.2 Auto-Recovery

If the background tracking engine crashes:

- ETS restarts the screenshot, idle detection, and sync timers automatically
- This ensures tracking continues without manual intervention

12.2.3 Fatal Errors

If a fatal error prevents ETS from running:

- The application logs the issue in `app_logs`
- Advises user to restart the app or PC
- If persistent, IT/Support must be contacted

12.3 Support Workflow

Clear workflows help resolve issues quickly.

12.3.1 Employee Support Steps

1. Check tray icon color
2. Reopen Dashboard
3. Restart ETS
4. Restart computer
5. Contact internal support with exported logs

12.3.2 Support Team Workflow

1. Review exported logs or directly inspect `app_logs`
2. Identify category of error:
    - Screenshot failure
    - Upload failure
    - Idle tracking failure
    - Authentication or server issue
3. Use logs to trace root cause
4. If required, update ETS to fix recurring issues
5. Communicate resolution to HR/Management

12.4 Future Enhancements

Planned or potential improvements that can be added in future releases:

12.4.1 Platform Expansion

- macOS client
- Linux desktop support

12.4.2 Auto-Update System

- Automatic updates delivered through backend
- Silent or prompt-based updates

12.4.3 Advanced Reporting (Server-Side)

- Graphical daily or weekly productivity view
- Idle trends
- Team or department-level dashboards

12.4.4 Offline Mode Enhancements

- More robust sync engine for long offline periods
- Additional retries and batching

12.4.5 Screenshot Optimization

- Adjustable screenshot resolution
- Adjustable image quality (compression)

12.4.6 Background Service Mode

- Run ETS as a Windows service for higher reliability

12.5 Maintenance Policy

12.5.1 Regular Updates

The application should receive updates for:

- Bug fixes
- Security improvements
- Feature enhancements
- Policy changes (HR/Management input)

12.5.2 Backend Maintenance

Backend must be periodically maintained:

- Database cleanup
- Logs archiving
- Screenshot purging (old data)
- Monitoring disk space on the server

12.5.3 Compatibility Maintenance

ETS must stay compatible with:

- Future Windows updates
- Python runtime changes
- API changes on the backend

12.5.4 Support Escalation

If ETS encounters a serious technical failure:

- Developer (Aushutosh Kumar) handles debugging
- Tech Lead (Shailabh Suman) reviews and approves fixes
- HR and CEO are informed only for policy-impacting issues

13. Appendices

This section contains supplementary material that supports the main specification. These items help clarify terminology, reference related documents, include additional diagrams, and track changes over time.

13.1 Glossary

A list of important terms used throughout the ETS documentation.

Term	Description
ETS	Employee Tracking System – the desktop app tracking screenshots and activity.
Screenshot Event	A randomly captured image of the employee's screen, encrypted and stored.
Idle Period	Time when no keyboard or mouse input is detected.
Session	A continuous period during which an employee is logged into ETS.

Term	Description
Shift	Defined work time for an employee (start and end in IST).
AES	Advanced Encryption Standard used for screenshot and data encryption.
SQLite	Local database engine used to store session, screenshot, idle log metadata.
System Tray	Windows taskbar area where ETS icon appears.
Encrypted File (.bin)	Encrypted screenshot file stored locally.
Config Sync	Regular fetch of updated settings from the server.
Idle Tracker	Component that detects user inactivity.
Sync Manager	Module that uploads screenshot and idle logs to the server.
IST	Indian Standard Time – used for all timestamps.

13.2 Reference Documents

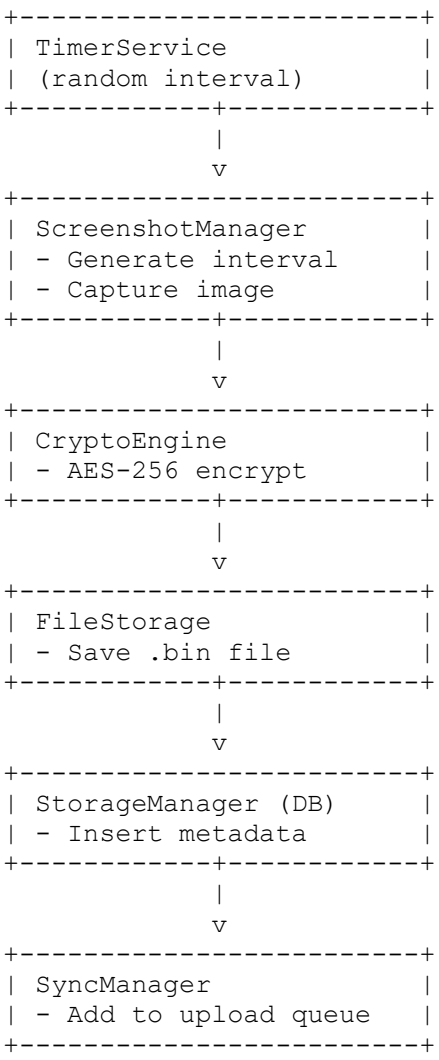
These documents may support the overall project but are not included directly in this file.

1. **Amaze Internet Services Pvt Ltd IT Policy**
   - User monitoring guidelines
   - Privacy policy
   - Data retention policy
2. **Amaze Internet Backend API Security Guidelines**
   - Token handling
   - HTTPS enforcement
   - Log sanitization
3. **Windows Desktop Application Design Principles**
   - Microsoft UI guidelines
   - Accessibility references
4. **Python Development Standards**
   - PEP8 coding style
   - Virtual environment usage
   - Packaging standards (PyInstaller)
5. **Data Privacy & Compliance Documents**
   - Employee monitoring compliance (internal)
   - Screenshot retention guidelines

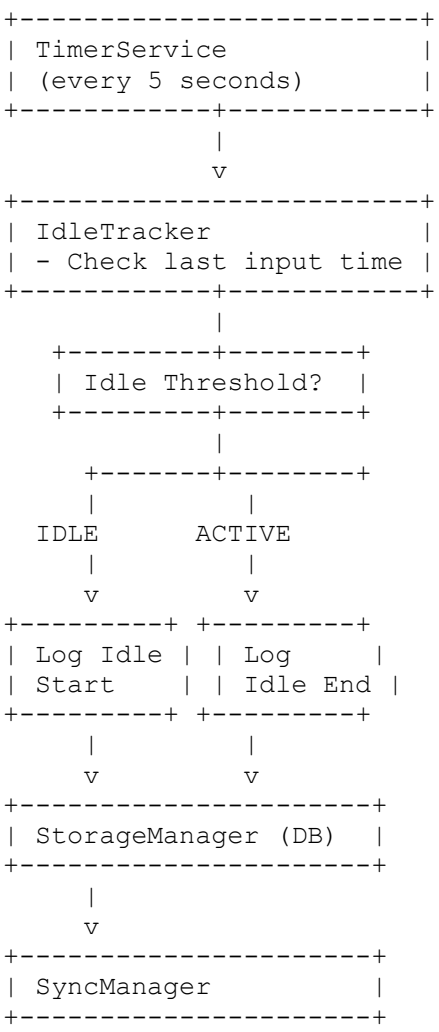
13.3 Additional Diagrams

Below are extended architecture diagrams and flows for deeper understanding.

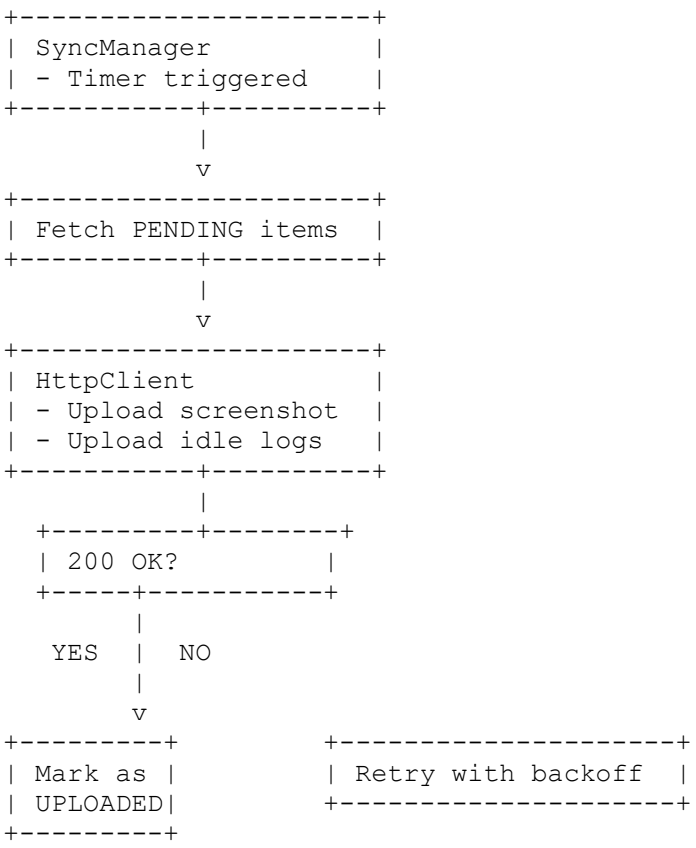
13.3.1 Screenshot Capture Workflow (Extended)



13.3.2 Idle Detection Workflow (Extended)



13.3.3 Upload Workflow (Extended)



13.4 Change Log

A record of all major modifications to the ETS documentation.

Version	Date	Description	Updated By
1.0 (Draft)	[DD/MM/YYYY]	Initial creation of ETS Technical Specification & Design Document	Shailabh Suman
1.1	—	—	—
1.2	—	—	—